Compte-Rendu : Travaux Pratiques de Blockchain

Matière : Blockchain & Technologies Décentralisées

TP 2 : Création d'une blockchain avec Python

Étudiant : Ahmed Channoufi

*Ce document présente les démarches, le code source, les sorties d'exécution ainsi que les analyses des **trois premiers exercices** du TP2.*

Exercice 1 : Découverte de la fonction de hachage SHA-256

Objectif : Comprendre le fonctionnement d'une fonction de hachage cryptographique (SHA-256), observer la taille de son empreinte et mettre en évidence l'effet d'avalanche.

1. Code Python

```
import hashlib

# 1. Calcul d'un premier hash
texte = "Blockchain"
hash_texte = hashlib.sha256(texte.encode("utf-8")).hexdigest()

print("Texte :", texte)
print("Hash  :", hash_texte)

# 2. Tests de variations
texte2 = "blockchain"
hash2 = hashlib.sha256(texte2.encode("utf-8")).hexdigest()

texte3 = "Blockchain et cryptographie"
hash3 = hashlib.sha256(texte3.encode("utf-8")).hexdigest()
```

2. Résultat dans le terminal

```
Texte : Blockchain
Hash  : 625da44e4eaf58d61cf048d168aa6f5e492dea166d8bb54ec06c30de07db57e1

Texte : blockchain
Hash  : ef7797e13d3a75526946a3bcf00daec9fc9c9c4d51ddc7cc5df888f74dd434d1

Texte : Blockchain et cryptographie
Hash  : f8fc99c0038743eb1434318e40676c5097187dea6147bbfcb4a67f3ec92ab18d
```

3. Réponses aux questions

Q1. Deux textes différents donnent-ils le même hash ?

Non, deux entrées distinctes génèrent des empreintes complètement différentes. Le simple passage d'un "B" majuscule à un "b" minuscule modifie entièrement le hash (propriété d'avalanche).

Q2. Quelle est la longueur d'un hash SHA-256 en caractères hexadécimaux ?

Il fait exactement **64 caractères hexadécimaux**, ce qui correspond à 256 bits (32 octets).

Q3. La longueur du texte d'origine influence-t-elle la longueur du hash ?

Non. Que le texte fasse un seul mot ou une phrase entière, la sortie produite par SHA-256 a toujours une longueur fixe de 64 caractères.

Q4. Pourquoi le hachage est-il indispensable dans une blockchain ?

Il fournit une empreinte infalsifiable du contenu de chaque bloc. Toute modification ultérieure, même d'un seul caractère, modifie son empreinte et rend la fraude immédiatement visible.

Q5. Peut-on retrouver facilement le texte initial à partir du hash ?

Non, SHA-256 est une fonction à sens unique (résistance à la préimage). Il est impossible par calcul direct de déduire le texte source à partir du hash.

Exercice 2 : Création du premier bloc (Classe Bloc)

Objectif : Modéliser un bloc sous forme de classe orientée objet, encapsuler ses propriétés (index, transaction, previous_hash) et automatiser le calcul de son empreinte.

1. Code Python

```
import hashlib

class Bloc:
    def __init__(self, index, transaction, previous_hash):
        self.index = index
        self.transaction = transaction
        self.previous_hash = previous_hash
        # Calcul automatique a la creation
        self.hash = self.calculer_hash()

    def calculer_hash(self):
        contenu = str(self.index) + "|" + self.transaction + "|" + self.previous_hash
        return hashlib.sha256(contenu.encode("utf-8")).hexdigest()

# Instanciation du premier bloc
bloc1 = Bloc(1, "Alice envoie 10 euros a Bob", "0")

# Test de modification
bloc1_modifie = Bloc(1, "Alice envoie 100 euros a Bob", "0")
```

2. Résultat dans le terminal

```
--- Informations du Bloc 1 ---
Index           : 1
Transaction     : Alice envoie 10 euros a Bob
Hash precedent  : 0
Hash actuel     : 0235e2abe7e4bf2794746aabba340a82de46da1f07b64ec08e7bd175ce1d1c85

--- Apres modification de la transaction (10 -> 100 euros) ---
Hash original : 0235e2abe7e4bf2794746aabba340a82de46da1f07b64ec08e7bd175ce1d1c85
Hash modifie  : dbb5eaea6cf6adeadf26a169376384c15b916826084da502ad627f503370a76c
Les deux hash sont identiques ? False
```

3. Réponses aux questions

Q1. Quel est le rôle de la méthode __init__() ?

C'est le constructeur de la classe. Il initialise les attributs du bloc lors de l'instanciation (index, transaction, previous_hash) et déclenche le calcul du hash.

Q2. Quel est le rôle de l'instruction self.hash = self.calculer_hash() ?

Elle calcule et affecte automatiquement l'empreinte cryptographique dès qu'un bloc est créé.

Q3. Pourquoi convertir l'index avec str(self.index) ?

L'index est un entier numérique. Pour le concaténer avec des chaînes de caractères et l'encoder en octets utf-8, la conversion en chaîne est requise.

Q4. Quels attributs participent au calcul du hash ?

L'index, la transaction et le previous_hash, séparés par un délimiteur ("|").

Q5. Que constate-t-on après modification de la transaction ?

Le hash résultant est complètement différent, ce qui prouve qu'une transaction falsifiée produit immédiatement une empreinte non concordante.

Exercice 3 : Chaînage cryptographique de trois blocs

Objectif : Assembler une chaîne ordonnée de blocs reliés par leurs empreintes et vérifier la cohérence du chaînage.

1. Code Python

```
# Bloc 1 (Genesis) : aucun predecesseur -> previous_hash = "0"
bloc1 = Bloc(1, "Alice envoie 10 euros a Bob", "0")

# Bloc 2 : reference le hash du bloc 1
bloc2 = Bloc(2, "Bob envoie 5 euros a Charlie", bloc1.hash)

# Bloc 3 : reference le hash du bloc 2
bloc3 = Bloc(3, "Charlie envoie 2 euros a Alice", bloc2.hash)

# Stockage sous forme de liste
blockchain = [bloc1, bloc2, bloc3]

# Affichage et verification
for bloc in blockchain:
    print(f"Bloc {bloc.index} | Prev: {bloc.previous_hash[:16]}... | Hash: {bloc.hash[:16]}...")
```

2. Résultat dans le terminal

```
------------------------------
Bloc          : 1
Transaction   : Alice envoie 10 euros a Bob
Hash precedent : 0
Hash actuel   : 0235e2abe7e4bf2794746aabba340a82de46da1f07b64ec08e7bd175ce1d1c85
------------------------------
Bloc          : 2
Transaction   : Bob envoie 5 euros a Charlie
Hash precedent : 0235e2abe7e4bf2794746aabba340a82de46da1f07b64ec08e7bd175ce1d1c85
Hash actuel   : 1e4e604e0e5727b738a61b5000b09df279568f0648faa5108c7e74ae958aa9e3
------------------------------
Bloc          : 3
Transaction   : Charlie envoie 2 euros a Alice
Hash precedent : 1e4e604e0e5727b738a61b5000b09df279568f0648faa5108c7e74ae958aa9e3
Hash actuel   : 4387302c6a3f17e6ef373507125256e75b8bb2c8ea61d096a0c134b5f93458c6

Verification des liaisons :
Bloc 2 prev_hash == Bloc 1 hash ? True
Bloc 3 prev_hash == Bloc 2 hash ? True
```

3. Réponses aux questions

Q1. Comment les blocs sont-ils reliés entre eux ?

Chaque bloc stocke dans son champ `previous_hash` l'empreinte exacte du bloc qui le précède directement.

Q2. Pourquoi le Bloc 2 utilise-t-il bloc1.hash ?

Pour créer le premier lien cryptographique de la chaîne, ancrant le Bloc 2 sur l'état du Bloc 1.

Q3. Pourquoi le Bloc 3 utilise-t-il bloc2.hash ?

Pour poursuivre le chaînage séquentiel : chaque nouveau bloc dépend du hash du bloc précédent.

Q4. Quel est le rôle de la liste 'blockchain' ?

Elle modélise le registre distribué sous forme de séquence ordonnée en mémoire.

Q5. Que se passe-t-il si on remplace bloc1.hash par '12345' pour le Bloc 2 ?

Le lien cryptographique est brisé car `previous_hash` ne correspond plus au hash réel du Bloc 1. La chaîne devient incohérente.